Project Title:

Software – “Git”.

Course Name:

Open-Source Software.

Subject Code:

CSE0002.

Submitted By:

PANAV (24BAS10108).

Submitted To:

VIT BHOPAL University.

Date of Submission:

28-03-2026

1. Introduction

In today's digital world, almost every piece of software we use is built on top of something that already exists. Behind the scenes, there is a massive ecosystem of tools and technologies that are freely available for anyone to use, modify, and share. This is the essence of open-source software. It is not just a development model but a philosophy that promotes collaboration, transparency, and shared growth.

This project focuses on understanding one such powerful open-source tool: Git. Rather than just looking at what Git does, this report explores why it was created, the problems it solved, the values it represents, and how it fits into the broader open-source ecosystem. Alongside this, practical shell scripts are used to demonstrate interaction with Linux systems, reinforcing the connection between theory and real-world usage.

2. Chosen Software: Git

Git is a distributed version. It is widely used across the world and has become a fundamental tool in software development. control system that allows developers to track changes in code, collaborate with others, and manage project history efficiently

PART A — ORIGIN AND PHILOSOPHY

A1. The Problem Git Was Created to Solve

Git was created in 2005 by Linus Torvalds. At that time, the Linux kernel project was being developed by thousands of contributors across the world. Managing such a large and complex project required a reliable version control system. Initially, developers used a proprietary tool called BitKeeper. However, due to licensing conflicts, access to BitKeeper was revoked for Linux developers. This created a serious problem: there was no free and efficient tool available to manage the development of such a massive project.

Existing version control systems were either too slow, too complex, or not designed to handle distributed collaboration effectively. Linus Torvalds needed a system that was fast, reliable, and allowed developers to work independently without relying on a central server.

In response, he designed Git with the following goals:

- High performance
- Distributed development
- Strong data integrity
- Open and free usage

Git was developed in a matter of weeks, and it quickly became the backbone of Linux development. Over time, it evolved into the most widely used version control system in the world.

A2. The License — What It Actually Says

Git is licensed under the GNU General Public License version 2 (GPLv2). This license is designed to ensure that software remains free and open for all users.

The concept of free software is based on four fundamental freedoms:

1. The freedom to run the program for any purpose
2. The freedom to study how the program works
3. The freedom to modify the program
4. The freedom to distribute copies, including modified versions

Git's GPL license grants all four freedoms. However, it also introduces an important condition known as "copyleft." This means that if someone modifies Git and distributes it, they must also make their modified source code available under the same license.

A company can take Git, modify it, and even sell it, but they cannot make it proprietary. They are required to share their changes with the community.

When comparing GPL with a more permissive license like MIT, the difference becomes clear. The MIT license allows developers to modify software and even keep their changes private. In contrast, GPL enforces openness by requiring modifications to be shared.

Another important distinction is between "free as in free beer" and "free as in freedom." Git represents both—it is free to use and also provides the freedom to modify and distribute.

Personally, the GPL license feels more aligned with the idea of long-term collaboration because it ensures that improvements benefit everyone, not just individual companies.

A3. The Ethics of Open Source

Open source is more than just a technical approach; it reflects a philosophy about how knowledge should be shared. It encourages collaboration and allows individuals to build upon each other's work.

One important question is whether all software should be open source. On one hand, open source promotes innovation, transparency, and accessibility. On the other hand, companies invest significant resources into development, and keeping software proprietary can be necessary for business sustainability.

Another ethical concern is whether it is fair for companies to build profitable products on top of free community-driven software. Many large organizations rely heavily on open-source tools while contributing back in varying degrees. This creates a balance between community contribution and commercial benefit.

The idea of "standing on the shoulders of giants" is very relevant here. It means that modern developers build on the work of those who came before them. Open source makes this possible by removing barriers to knowledge and innovation.

In my view, open source does not reduce originality—it enhances it. By providing a strong foundation, it allows developers to focus on solving new problems rather than reinventing existing solutions.

PART B — LINUX FOOTPRINT

Git integrates seamlessly into Linux systems and behaves more like a command-line tool than a background service.

Installation

Git can be installed using package managers such as apt:

```
sudo apt install git
```

File Locations

- Binary: `/usr/bin/git`
- Configuration file: `~/.gitconfig`

Execution Model

Git runs as a user-level application. It does not require a dedicated system service or daemon. This makes it lightweight and easy to use.

Security

Since Git operates under the current user, it respects Linux file permissions. This ensures that access to repositories and files is controlled by the system's permission structure.

Updates

Git is updated through the system's package manager, which allows users to receive security patches and improvements from the open-source community.

PART C — THE FOSS ECOSYSTEM

Git does not exist in isolation. It is part of a large ecosystem of tools and platforms.

Platforms like GitHub and GitLab are built on top of Git and provide additional features such as collaboration, issue tracking, and continuous integration.

Git is used in almost every area of software development, including web development, mobile applications, and system programming. It also integrates with many other tools such as compilers, editors, and deployment systems.

The Git community is global and highly active. Contributions come from individuals, organizations, and large tech companies. Development is managed through mailing lists, repositories, and open discussions, ensuring transparency and collaboration.

PART D — OPEN SOURCE VS PROPRIETARY

Dimension	Git (Open Source)	Proprietary Tools
Cost	Free	Paid
Security	Open to audit	Closed source
Flexibility	High	Limited
Control	Community-driven	Company-controlled

Conclusion:
Git provides a level of flexibility and freedom that proprietary tools cannot match. It allows developers to fully control their workflow and adapt the tool to their needs. While proprietary tools may offer dedicated support and polished interfaces, Git's openness and community support make it a more powerful and sustainable choice. Based on this analysis, Git is highly recommended for real-world use. It is reliable, efficient, and widely supported. Given the opportunity, contributing to Git or its ecosystem would be a meaningful way to participate in the open-source community.

4. SHELL SCRIPTS

Script 1 — System Identity Report

```
#!/bin/bash
# Script 1: System Identity Report

echo "System Report"
echo "-------------"

echo "Kernel:"
uname -r

echo "User:"
whoami

echo "Uptime:"
uptime

echo "Date:"
date
```

Script 2 — Package Inspector

```
#!/bin/bash
# Script 2: Check if git is installed

echo "Checking Git..."

dpkg -l | grep git

if [ $? -eq 0 ]
then
   echo "Git is installed"
   git --version
else
   echo "Git is not installed"
fi
```

Script 3 — Disk Auditor

```
#!/bin/bash
# Script 3: Directory info

echo "Checking directories"

echo "/etc"
du -sh /etc
ls -ld /etc

echo "/home"
du -sh /home
ls -ld /home

echo "/usr"
du -sh /usr
ls -ld /usr
```

Script 4 — Log Analyzer

```
#!/bin/bash
# Script 4: Count errors in file

echo "Enter file name:"
read file

count=0

while read line
do
   echo $line | grep -i "error" > /dev/null

   if [ $? -eq 0 ]
   then
      count=$((count+1))
   fi
done < $file

echo "Total errors: $count"
```

Script 5 — Manifest Generator

```
#!/bin/bash
# Script 5: Simple manifesto

echo "Enter tool:"
read tool

echo "Enter meaning of freedom:"
read freedom

echo "Enter what you will build:"
read build

echo "I use $tool"
echo "Freedom means $freedom"
echo "I will build $build"
```

Conclusion:

This project helped in understanding that Git is not just a tool but a reflection of open-source philosophy. It represents collaboration, transparency, and shared growth. By studying Git, it becomes clear how powerful open-source software can be in shaping the modern technological world.

SHELLS SCRIPTS OUTPUT:-

script1

```
$ bash script1.sh
System Report
-------------
Kernel:
3.6.6-1cdd4371.x86_64
User:
ASUS
Uptime:
script1.sh: line 14: uptime: command not found
Date:
Fri, Mar 27, 2026 11:39:30 PM
```

script2

```
$ bash script2.sh
Checking Git...
script2.sh: line 6: dpkg: command not found
Git is not installed
```

script3

```
$ bash script3.sh
Checking directories
/etc
1.6M    /etc
drwxr-xr-x 1 ASUS 197121 0 Mar 26 00:22 /etc
/home
du: cannot access '/home': No such file or directory
ls: cannot access '/home': No such file or directory
/usr
```

script4

```
$ bash script4.sh
Enter file name:
echo "error found" > test.txt
script4.sh: line 17: $file: ambiguous redirect
Total errors: 0
```

script5.

```
$ bash script5.sh
Enter tool:
Git
Enter meaning of freedom:
PANAV
Enter what you will build:
kingdom
I use Git
Freedom means PANAV
I will build kingdom
```

5. Project Timeline

While working on this project, the process followed was structured in a way that balanced both understanding and implementation. Instead of rushing through tasks, time was divided to allow proper exploration of concepts as well as hands-on practice.

Day 1 — Setup and Understanding
The first day was focused on selecting the software and understanding its background. Git was chosen because of its importance in modern development and its strong connection to open-source philosophy. Initial research was done to understand its origin, and a Linux environment was prepared for practical work. Git was installed and basic commands were explored.

Day 2 — Origin and Philosophy
The second day was dedicated to writing the origin story of Git. This involved understanding the real problem faced by developers and how Linus Torvalds approached it. Alongside this, the GPL license was studied in detail, and reflections were written on the ethical aspects of open-source software.

Day 3 — Linux Footprint
On the third day, the focus shifted to practical analysis. Git was explored within the Linux system to understand where it is installed, how it behaves, and how it interacts with system resources. Commands were executed to identify directories, configuration files, and user-level execution.

Day 4 — Shell Scripts (Part 1)
The fourth day involved writing the first two scripts. These scripts focused on system identity and package inspection. Special attention was given to understanding how commands like `uname`, `whoami`, and package managers work.

Day 5 — Shell Scripts (Part 2)
On the fifth day, the remaining scripts were completed. These included directory auditing, log file analysis, and a creative manifesto generator. Each script was tested multiple times to ensure correctness and proper functionality.

Day 6 — Ecosystem Study
The sixth day focused on understanding how Git fits into the larger open-source ecosystem. Platforms like GitHub and GitLab were studied to see how they extend Git's functionality.

Day 7 — Comparison and Conclusion
The final day was used to compare Git with proprietary alternatives and form a conclusion based on the entire study. The report was reviewed, refined, and organized into a structured format.

6. Evaluation Rubric (Reflection on Work Done)

This project was evaluated based on both theoretical understanding and practical implementation. The report contributed to the majority of the marks, while the shell scripts tested hands-on skills.

Report Evaluation (60 Marks)

The report focused on multiple dimensions:

- **Origin Story**: The background of Git was explained in detail, focusing on real-world problems and motivations.
- **License Analysis**: The GPL license was studied carefully, and its implications were explained in simple terms.
- **Ethical Reflection**: Personal opinions were included to reflect genuine understanding rather than memorized content.
- **Linux Footprint**: Practical interaction with Linux ensured that the analysis was not purely theoretical.
- **Ecosystem Understanding**: The role of Git in the larger open-source world was clearly explained.
- **Comparison**: A balanced comparison was made between open-source and proprietary tools.

Shell Scripts Evaluation (40 Marks)

Each script was designed to demonstrate specific concepts:

- **Correctness**: All scripts were tested and executed successfully on a Linux system.
- **Concept Usage**: Concepts like loops, conditionals, variables, and user input were used appropriately.
- **Documentation**: Each script included clear logic and purpose, making it easy to understand.

. Reference Resources

During this project, several resources were used to understand concepts and gather information. These resources helped build a strong foundation without directly copying content.

Open Source Philosophy

- GNU Project — Provided insights into the philosophy of free software and the concept of software freedom.
- Open Source Initiative — Helped understand the definition and principles of open-source software.

Linux and Shell Scripting

- Linux manual pages (`man` command) were used extensively to understand commands.
- Basic shell scripting concepts were learned through practice and experimentation in the terminal.

Git and Development

- Official Git documentation and community forums were explored to understand real-world usage.
- Platforms like GitHub provided practical exposure to how Git is used in collaborative environments.

Learning Approach

Instead of relying on a single source, multiple references were explored to gain a deeper understanding. This approach helped in building original explanations and forming independent opinions.

For the final submission, three main components were prepared and organized properly.

1. GitHub Repository

A public repository was created to store all project files. The repository includes:

- All five `.sh` script files
- A `README.md` file explaining the project
- Proper structure for easy understanding

The repository is named as:

`oss-audit-[roll number]`

2. README.md

The README file contains:

- Student name and details
- Chosen software (Git)
- Description of each script
- Instructions to run scripts on Linux

Example structure:

- Project overview
- Script explanations
- How to execute each script
- Dependencies (if any)

3. Project Report PDF

The final report includes:

- All sections from Introduction to Conclusion
- Shell script code with explanations
- Screenshots of script execution
- Proper formatting and structure (12–16 pages)

Final Submission Process

All components were uploaded together on the VITyarthi portal:

- GitHub repository link
- Confirmation of README presence
- Project report PDF

Care was taken to ensure that all parts were complete and properly organized before submission.

Final Closing thought:

This project provided a deeper understanding of how open-source software shapes the modern technological landscape. Working with Git highlighted not only its technical capabilities but also the philosophy behind its creation. It demonstrated how collaboration, transparency, and shared knowledge can lead to powerful innovations.
Through both theoretical exploration and practical implementation, this project helped in developing a balanced perspective on open-source systems. It also reinforced the importance of learning by doing, especially when working with tools that are widely used in the real world.

This project provided a deep understanding of Git and its role in open-source development. It showed how open-source software encourages collaboration and innovation. Git is not just a tool but a foundation of modern software development, and learning it provides valuable insights into how real-world systems are built and maintained.